```
// This works too; outputs: I'd like an Alexander Keith's
echo "I'd like an {${beers::$ale}}\n";
?>
```

4.2.3 String Access and Modification by Character

Characters within strings may be accessed and modified by specifying the zero-based offset of the desired character after the string using square array brackets, as in *$str*. Think of a string as an array of characters for this purpose. The functions substr() and substr_replace() can be used when you want to extract or replace more than 1 character.

Keyword

Integer is a whole number (not a fraction) that can be positive, negative, or zero. Therefore, the numbers 10, 0, -25, and 5,148 are all integers. Unlike floating point numbers, integers cannot have decimal places. Integers are a commonly used data type in computer programming.

As of PHP 7.1.0, negative string offsets are also supported. These specify the offset from the end of the string. Formerly, negative offsets emitted E_NOTICE for reading (yielding an empty string) and E_WARNING for writing (leaving the string untouched).

Writing to an out of range offset pads the string with spaces. Non-integer types are converted to **integer**. Illegal offset type emits E_NOTICE. Only the first character of an assigned string is used. As of PHP 7.1.0, assigning an empty string throws a fatal error. Formerly, it assigned a NULL byte.

Internally, PHP strings are byte arrays. As a result, accessing or modifying a string using array brackets is not multi-byte safe, and should only be done with strings that are in a single-byte encoding such as ISO-8859-1.

Example #11. Some string examples

```
<?php
// Get the first character of a string
$str = 'This is a test.';
$first = $str[0];

// Get the third character of a string
$third = $str[2];
```